

Higher Order Functions

CSCI 1101

2024-04-30

Data

The types of things that programs can manipulate constitute **data**.

These are the the things that we can:

- assign to variables,
- pass to functions as arguments,
- return from functions as results,
- store in data structures such as lists and dictionaries,
- etc.

We have seen may examples of data in python programs, including:

booleans, integers, strings, lists, sets, dictionaries, exceptions, classes.

Functions Are Data

Functions are data too!

To treat a function as data, simply *refer to* it without *calling* it:

```
> print('hello')           # calling a function
```

```
hello
```

```
> print                     # referring to a function
```

```
<built-in function print>
```

Treating Functions as Data

We can do with functions the things that we can do with other kinds of data, including:

assign them to variables:

```
> fred = print
```

store them in lists:

```
> freds = [fred, fred, fred]
```

print them:

```
> fred(fred)
```

```
<built-in function print>
```

Function Literals

We can write expressions for function values, called **function literals** (for historical reasons they are also called “*lambda expressions*”):

```
lambda <parameter_variables> : <any_expression>
```

- A function literal that doubles a number:

```
double = lambda num : 2 * num
```

- A function literal that tests whether a number is even:

```
is_even = lambda num : num % 2 == 0
```

- A function literal that adds two numbers:

```
add = lambda x , y : x + y
```

Activity: Function Literals

Write python function literal expressions for the following functions:

- $4x$
- x^2
- $x * y$

Higher-Order Functions

One of the most useful things that we can do with functions is pass them as arguments to other functions.

A function that accepts a function as an argument is a **higher-order function**.

```
def call(function , argument) :  
    ''' sig: (fun (a -> b) , a) --> b '''  
    return function(argument)
```

This function takes a function and an element of that function's argument type and applies the function to that element.

```
> call(print , 'hello')  
hello
```

```
> call(double , 21)  
42
```

Mapping

Recall the *mapping pattern*, which takes a function `f` and a list `items` and applies the function to each element of the list:

```
acc = []
for item in items :
    acc.append(f(item))
```

We can turn this pattern into a higher-order function by just adding two lines:

```
def list_map(f , items) :
    ''' sig: (fun (a -> b) , list a) --> list b '''
    acc = []
    for item in items :
        acc.append(f(item))
    return acc
```


Python's Map Function

This function is already included in the python standard library.

It is called `map`, and returns an *iterator* object, which you can listify if needed.

```
> list(map(double , range(6)))  
[0, 2, 4, 6, 8, 10]
```

```
> list(map(is_even , range(6)))  
[True, False, True, False, True, False]
```

```
> list(map(lambda n : n + 1 , range(6)))  
[1, 2, 3, 4, 5, 6]
```

Activity: Initials

Use python's `lambda` expressions and `map` function to write an expression `initials` that extracts the first characters from a list of strings:

```
> list(initials(['it', 'was', 'the', 'best', 'of', 'times']))  
['i', 'w', 't', 'b', 'o', 't']
```

Your expression should not contain any of the keywords `def`, `for`, or `while`.

Filtering

Recall the *filtering pattern*, which takes a predicate `p` and a list `items` and returns the list of elements that satisfy the predicate:

```
acc = []  
for item in items :  
    if p(item) :  
        acc.append(item)
```

We can turn this pattern into a higher-order function by just adding two lines:

```
def list_filter(p , items) :  
    ''' sig: (fun (a -> bool) , list a) --> list a '''  
    acc = []  
    for item in items :  
        if p(item) :  
            acc.append(item)  
    return acc
```

Python's Filter Function

This function is already included in the python standard library.

It is called `filter`, and returns an *iterator* object, which you can listify if needed.

```
> list(filter(is_even , range(6)))  
[0, 2, 4]
```

```
> ''.join(filter(str.isupper , list('HeLlO WoRlD')))  
'HLOWRD'
```

```
> list(filter(lambda n : n >= 3 , range(6)))  
[3, 4, 5]
```

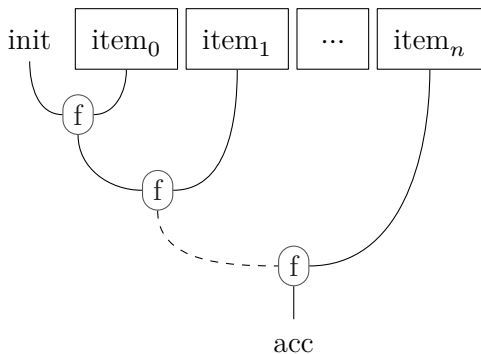
The Accumulator Pattern

Mapping and filtering are two instances of the *accumulator pattern*.

In the general case, we provide an accumulator *initial value* `init` and *update function* `f`, and use them to build a *result value* out of a list `items`:

```
acc = init
for item in items :
    acc = f(acc , item)
```

Conceptually, this does the following:



List Reducing

Again, we can turn this pattern into a higher-order function by just adding two lines:

```
def list_reduce(f , init , items) :  
    ''' sig: (fun (b , a -> b) , b , list a) --> b '''  
    acc = init  
    for item in items :  
        acc = f(acc , item)  
    return acc
```

This lets us turn the whole the accumulator pattern into a single expression:

```
> list_reduce(add , 0 , [1, 2, 3, 4, 5])  
15
```

```
> list_reduce((lambda n,s:n+len(s)) , 0 , ['hello','world'])  
10
```

Why Use Higher Order Functions?

By now you are probably very familiar with writing `for`-loops.

Like recursion, higher-order functions make you think about problem solving in a different way.

This may feel unfamiliar at first, but it is worthwhile. Here are a few reasons:

Efficiency: Repeating *boilerplate code* over and over is a waste of human time and ingenuity. Just turn it into a function and call it when you need it.

Abstraction: This lets you work at a higher level of conceptual abstraction, focusing on *what* you want to accomplish, rather than *how*.

Optimization: It also lets the computer run your programs more efficiently, for example, by *parallelizing* independent operations.

To Do

Finish *Homework 12: More Object-Oriented Programming* on CodeRunner by Thursday.

Finish *Project 4: Bioinformatics* on CodeRunner in by 2024-05-08.